2.2. Các mức độ kiểm thử (test types) và Loại kiểm thử (test levels)

Test Levels (Mức độ kiểm thử):

- Các nhóm hoạt động kiểm thử được tổ chức và quản lý cùng nhau
- Mỗi mức độ = một **thực thể của quy trình kiểm thử**
- Tương ứng với phần mềm ở một **giai đoạn phát triển cụ thể**
- Từ thành phần riêng lẻ → toàn bộ hệ thống → hệ thống của các hệ thống

Test Types (Loại kiểm thử):

- Các nhóm hoạt động kiểm thử liên quan đến **đặc tính chất lượng cụ thể**
- Có thể thực hiện ở **MỌI mức độ kiểm thử**

2.2.1. Các mức độ kiểm thử

Trong giáo trình này, năm mức độ kiểm thử sau đây được mô tả:

- Kiểm thử thành phần (Component testing - còn gọi là kiểm thử đơn vị)
- Kiểm thử tích hợp thành phần (Component integration testing - còn gọi là kiểm thử tích hợp đơn vị)
- Kiểm thử hệ thống (System testing)
- Kiểm thử tích hợp hệ thống (System integration testing)
- Kiểm thử chấp nhận (Acceptance testing)

test levels

Kiểm thử thành phần (Component testing - còn gọi là kiểm thử đơn vị)

- Tập trung vào việc kiểm thử các thành phần trong sự cô lập. Nó thường yêu cầu sự hỗ trợ cụ thể như khung kiểm thử đơn vị (unit test frameworks) và thông thường do **lập trình viên thực hiện** trong môi trường phát triển của họ.

Mô tả: Kiểm thử các thành phần **trong sự cô lập** (isolation)

📊 **Bảng đặc điểm đầy đủ:**

Thuộc tính	Chi tiết
Test Object	Thành phần riêng lẻ: hàm, class, module
Test Objectives	✓ Giảm rủi ro ✓ Xác minh hành vi chức năng/phi chức năng ✓ Xây dựng niềm tin vào chất lượng thành phần ✓ Tìm lỗi trong thành phần ✓ Ngăn lỗi lan ra mức độ cao hơn
Test Basis	✓ Thiết kế chi tiết (Detailed design) ✓ Code (Source code) ✓ Data model ✓ Component specifications
Test Objects	✓ Thành phần, đơn vị, module ✓ Code và cấu trúc dữ liệu ✓ Class, Database modules
Typical Defects	✓ Logic không chính xác (Incorrect logic) ✓ Vấn đề luồng dữ liệu (Data flow issues) ✓ Code không đúng/thiếu (Incorrect code)
Approach	✓ Thường là White-box testing ✓ Có thể có Test-Driven Development (TDD) ✓ Sử dụng code coverage
Responsibilities	✓ Developers (chủ yếu) ✓ Trong môi trường phát triển của họ
Tools Required	✓ Unit test frameworks (JUnit, NUnit, pytest, Jest...) ✓ Mocking/Stubbing tools ✓ Code coverage tools (JaCoCo, Coverage.py...)
Environment	Development environment (local machine)

💡 **Ví dụ chi tiết:**

```
// Component: calculateDiscount() - Hàm tính giảm giá
function calculateDiscount(price, discountPercent, customerType) {
  if (price < 0 || discountPercent < 0 || discountPercent > 100) {
    throw new Error('Dữ liệu không hợp lệ');
  }

  let discount = price * (discountPercent / 100);

  // Khách hàng VIP được giảm giá thêm 5%
  if (customerType === 'VIP') {
    discount += price * 0.05;
```

```
  }

  return Math.min(discount, price); // Giảm giá không thể vượt quá giá gố
}

// Component Tests (Unit Tests) - Kiểm thử thành phần (Kiểm thử đơn vị)
describe('calculateDiscount', () => {
  // Test 1: Chức năng cơ bản
  test('tính giảm giá 10% chính xác', () => {
    expect(calculateDiscount(100, 10, 'NORMAL')).toBe(10);
  });

  // Test 2: Giá trị biên
  test('xử lý giảm giá 0%', () => {
    expect(calculateDiscount(100, 0, 'NORMAL')).toBe(0);
  });

  // Test 3: Khách hàng VIP
  test('áp dụng giảm giá thêm cho VIP', () => {
    expect(calculateDiscount(100, 10, 'VIP')).toBe(15); // 10% + 5%
  });

  // Test 4: Xử lý ngoại lệ
  test('ném lỗi khi giá âm', () => {
    expect(() => calculateDiscount(-100, 10, 'NORMAL'))
      .toThrow('Dữ liệu không hợp lệ');
  });

  // Test 5: Trường hợp đặc biệt
  test('giảm giá không thể vượt quá giá gốc', () => {
    expect(calculateDiscount(100, 200, 'NORMAL')).toBe(100);
  });
});

// Mục tiêu kiểm thử đạt được:
// ✓ Xác minh hành vi chức năng (giảm giá được tính đúng)
// ✓ Tìm lỗi sớm (lỗi logic trong sự cô lập)
// ✓ Kiểm thử phi chức năng (xử lý ngoại lệ)
// ✓ Ngăn lỗi lan ra (phát hiện trước khi tích hợp)
```

Kiểm thử tích hợp thành phần (Component integration testing - còn gọi là kiểm thử tích hợp đơn vị):

- Tập trung vào việc kiểm thử các giao diện và tương tác giữa các thành phần. Mức độ này phụ thuộc nhiều vào chiến lược tích hợp như: từ dưới lên (bottom-up), từ trên xuống (top-down) hoặc tích hợp toàn bộ (big-bang).

Mô tả: Kiểm thử **giao diện và tương tác** giữa các thành phần

📊 **Bảng đặc điểm đầy đủ:**

Thuộc tính	Chi tiết
Test Object	Giao diện giữa các thành phần, subsystems
Test Objectives	✓ Giảm rủi ro ✓ Xác minh giao tiếp giữa các thành phần ✓ Tìm lỗi trong giao diện và tương tác ✓ Ngăn lỗi lan ra system level
Test Basis	✓ Software và system design ✓ Architecture ✓ Workflows ✓ Use cases
Test Objects	✓ Subsystems ✓ Databases ✓ Microservices ✓ APIs giữa các thành phần
Typical Defects	✓ Dữ liệu truyền không đúng/bị mất ✓ Giao diện API sai ✓ Gọi sai logic giữa modules ✓ Lỗi data format
Approach	✓ Phụ thuộc vào integration strategy ✓ Bottom-up: Test từ dưới lên ✓ Top-down: Test từ trên xuống ✓ Big-bang: Tích hợp tất cả cùng lúc
Responsibilities	✓ Developers hoặc Testers ✓ Tùy thuộc vào tổ chức

🔀 **3 chiến lược tích hợp chi tiết:**

1. Bottom-up Integration:

```
Tầng 3: UI Module
        ↑ (Test last)
Tầng 2: Business Logic
        ↑ (Test second)
Tầng 1: Database Module ← (Test first)
```

```
Ưu điểm:
✓ Không cần stub (vì test từ dưới lên)
✓ Dễ tạo test data
✓ Lỗi database được phát hiện sớm

Nhược điểm:
✗ UI/Critical features được test muộn
✗ Cần driver để test modules dưới
```

2. Top-down Integration:

```
Tầng 1: UI Module ← (Test first)
        ↓ (Test second)
Tầng 2: Business Logic
        ↓ (Test last)
Tầng 3: Database Module

Ưu điểm:
✓ UI/Critical features được test sớm
✓ Không cần driver
✓ Phù hợp với Agile (user-facing features trước)

Nhược điểm:
✗ Cần tạo nhiều stub cho modules dưới
✗ Lỗi database phát hiện muộn
```

3. Big-bang Integration:

```
Module A ─┐
Module B ─┼─> Tích hợp tất cả cùng lúc → Test
Module C ─┤
Module D ─┘

Ưu điểm:
✓ Nhanh, đơn giản (không cần test từng bước)

Nhược điểm:
✗ Khó xác định lỗi nằm ở module nào
✗ High risk (tất cả lỗi xuất hiện cùng lúc)
✗ Chỉ nên dùng cho hệ thống nhỏ
```

💡 **Ví dụ thực tế:**

```
// Component 1: ShoppingCart - Giỏ hàng
class ShoppingCart {
  constructor() {
    this.items = [];
  }

  addItem(product, quantity) {
    this.items.push({ product, quantity });
  }

  getTotal() {
    return this.items.reduce((sum, item) =>
      sum + (item.product.price * item.quantity), 0
    );
  }

  getItems() {
    return this.items;
  }
}

// Component 2: PaymentProcessor - Xử lý thanh toán
class PaymentProcessor {
  processPayment(amount, paymentMethod) {
    if (amount <= 0) {
      throw new Error('Số tiền không hợp lệ');
    }
    // Xử lý thanh toán...
    return { success: true, transactionId: '123' };
  }
}

// Component Integration Test - Kiểm thử tích hợp thành phần
describe('Tích hợp ShoppingCart với PaymentProcessor', () => {
  let cart;
  let paymentProcessor;

  beforeEach(() => {
    cart = new ShoppingCart();
    paymentProcessor = new PaymentProcessor();
  });

  test('xử lý thanh toán với tổng tiền giỏ hàng chính xác', () => {
    // Chuẩn bị
```

```
    cart.addItem({ name: 'iPhone', price: 1000 }, 2);
    cart.addItem({ name: 'Ốp lưng', price: 50 }, 1);

    const total = cart.getTotal(); // 2050

    // Thực hiện
    const result = paymentProcessor.processPayment(total, 'CREDIT_CARD');

    // Kiểm tra
    expect(result.success).toBe(true);
    expect(result.transactionId).toBeDefined();
  });

  test('thanh toán thất bại khi giỏ hàng trống', () => {
    // Chuẩn bị: Giỏ hàng trống
    const total = cart.getTotal(); // 0

    // Thực hiện & Kiểm tra
    expect(() => paymentProcessor.processPayment(total, 'CREDIT_CARD'))
      .toThrow('Số tiền không hợp lệ');
  });
});

// Mục tiêu kiểm thử đạt được:
// ✓ Xác minh giao diện giữa ShoppingCart và PaymentProcessor
// ✓ Tìm lỗi trong việc truyền dữ liệu (xác thực số tiền)
// ✓ Ngăn lỗi lan ra mức độ hệ thống
```

Kiểm thử hệ thống (System testing):

Tập trung vào **hành vi và khả năng tổng thể** của toàn bộ hệ thống hoặc sản phẩm, thường bao gồm kiểm thử chức năng cho các tác vụ đầu-cuối (end-to-end) và kiểm thử phi chức năng cho các đặc tính chất lượng. Đối với một số đặc tính phi chức năng, tốt nhất là nên kiểm thử trên một hệ thống hoàn chỉnh trong một môi trường kiểm thử đại diện (ví dụ: tính khả dụng). Kiểm thử hệ thống có thể do một **đội kiểm thử độc lập** thực hiện và liên quan đến các đặc tả của hệ thống.

Mô tả: Kiểm thử **hành vi và khả năng tổng thể** của toàn bộ hệ thống

📊 **Bảng đặc điểm đầy đủ:**

Thuộc tính	Chi tiết

Thuộc tính	Chi tiết
Test Object	Toàn bộ hệ thống hoàn chỉnh, applications, hardware/software systems
Test Objectives	✓ Giảm rủi ro ✓ Xác minh hành vi chức năng và phi chức năng ✓ Validate hoàn chỉnh end-to-end ✓ Xây dựng niềm tin vào chất lượng tổng thể
Test Basis	✓ System và software requirements ✓ Use cases ✓ Functional specifications ✓ Risk analysis reports
Test Objects	✓ Hệ thống, user và operation manuals ✓ System configuration và configuration data
Typical Defects	✓ Hành vi hệ thống không đúng/không mong đợi ✓ End-to-end flow không hoạt động ✓ Không đáp ứng requirements ✓ Lỗi phi chức năng (performance, security, usability...)
Approach	✓ Chủ yếu Black-box testing ✓ Cả Functional và Non-functional testing ✓ Test trong môi trường đại diện production
Responsibilities	✓ **Independent test team** (quan trọng!) ✓ Đội kiểm thử độc lập với dev team
Environment	✓ Test environment (gần giống production) ✓ Đầy đủ data, configurations ✓ Tốt nhất là giống hệt production

🎯 **Test Objectives chi tiết:**

1. Kiểm thử chức năng (Functional Testing - End-to-End):

```
Ví dụ: Hệ thống thương mại điện tử

E2E Test Case: Quy trình mua hàng hoàn chỉnh
1. Đăng ký người dùng
   → Điều hướng đến trang đăng ký
   → Điền thông tin người dùng
   → Xác thực email

2. Tìm kiếm sản phẩm
   → Tìm kiếm "iPhone 15"
   → Áp dụng bộ lọc (giá, thương hiệu)
   → Xem chi tiết sản phẩm
```

```
3. Giỏ hàng
   → Thêm sản phẩm vào giỏ
   → Cập nhật số lượng
   → Áp dụng mã giảm giá

4. Thanh toán
   → Nhập địa chỉ giao hàng
   → Chọn phương thức vận chuyển
   → Chọn phương thức thanh toán

5. Thanh toán
   → Nhập thông tin thanh toán
   → Xác nhận thanh toán
   → Nhận xác nhận

6. Theo dõi đơn hàng
   → Xem trạng thái đơn hàng
   → Nhận thông báo qua email

✓ Kiểm thử toàn bộ quy trình từ đầu đến cuối
✓ Tất cả modules hoạt động cùng nhau
✓ Dữ liệu chuyển đúng qua các modules
```

2. Non-functional Testing - Kiểm thử phi chức năng:

```
Kiểm thử hiệu năng (Performance Testing):
├─ Load Test: 1000 người dùng đồng thời
├─ Stress Test: Tìm điểm vỡ hệ thống
├─ Endurance Test: Chạy liên tục 24 giờ
└─ Spike Test: Tăng đột ngột 10000 người dùng

Kiểm thử bảo mật (Security Testing):
├─ SQL Injection
├─ XSS (Cross-Site Scripting)
├─ Vượt qua xác thực (Authentication bypass)
├─ Kiểm thử phân quyền (Authorization tests)
└─ Mã hóa dữ liệu (Data encryption)

Kiểm thử khả năng sử dụng (Usability Testing):
├─ Điều hướng có trực quan không?
├─ Thông báo lỗi có rõ ràng không?
```

```
├─ Thời gian phản hồi có chấp nhận được không?
└─ Có thân thiện với di động không?

Kiểm thử tương thích (Compatibility Testing):
├─ Trình duyệt: Chrome, Firefox, Safari, Edge
├─ Hệ điều hành: Windows, macOS, Linux
├─ Thiết bị: Desktop, Tablet, Mobile
└─ Độ phân giải màn hình: 1920x1080, 1366x768...
```

💡 **Ví dụ System Test Case:**

```
Mã test case: SYS_TC_001
Tiêu đề: Quy trình mua hàng thương mại điện tử hoàn chỉnh

Điều kiện tiên quyết:
- Dữ liệu test có sẵn trong cơ sở dữ liệu
- Môi trường test được cấu hình đúng
- Cổng thanh toán (sandbox) có thể truy cập

Các bước test:
1. Mở trình duyệt và điều hướng đến https://testshop.com
2. Nhấp "Đăng ký" và tạo tài khoản mới
   - Email: testuser@example.com
   - Mật khẩu: Test@123
3. Xác minh email đăng ký đã nhận
4. Đăng nhập bằng tài khoản mới
5. Tìm kiếm "iPhone 15 Pro"
6. Áp dụng bộ lọc: Giá < $1000
7. Nhấp vào sản phẩm đầu tiên
8. Xác minh chi tiết sản phẩm hiển thị đúng
9. Nhấp "Thêm vào giỏ hàng"
10. Nhấp biểu tượng giỏ hàng → Cập nhật số lượng thành 2
11. Nhập mã khuyến mãi "SAVE20"
12. Nhấp "Tiến hành thanh toán"
13. Nhập địa chỉ giao hàng:
    - 123 Đường Test, Thành phố Test, 10000
14. Chọn "Giao hàng tiêu chuẩn (3-5 ngày)"
15. Chọn phương thức thanh toán "Thẻ tín dụng"
16. Nhập thẻ test: 4111 1111 1111 1111
17. Nhấp "Đặt hàng"
18. Xác minh trang xác nhận đơn hàng
19. Kiểm tra email xác nhận đơn hàng
```

```
20. Điều hướng đến "Đơn hàng của tôi"
21. Xác minh đơn hàng mới xuất hiện với trạng thái "Đang xử lý"

Kết quả mong đợi:
✓ Người dùng đăng ký thành công
✓ Đăng nhập thành công
✓ Tìm kiếm trả về kết quả đúng
✓ Chi tiết sản phẩm chính xác
✓ Tổng giỏ hàng được tính đúng (có giảm giá)
✓ Phí giao hàng được áp dụng đúng
✓ Thanh toán xử lý thành công
✓ Email xác nhận đơn hàng đã gửi
✓ Đơn hàng hiển thị trong lịch sử đơn hàng
✓ Kho hàng đã cập nhật (tồn kho giảm 2)
✓ Thời gian phản hồi < 3 giây cho mỗi trang
✓ Không có lỗi trong console trình duyệt

Mục tiêu kiểm thử đã xác minh:
✓ Quy trình chức năng end-to-end hoạt động
✓ Tất cả thành phần hệ thống tích hợp đúng
✓ Hệ thống đáp ứng yêu cầu chức năng
✓ Yêu cầu phi chức năng được đáp ứng (hiệu năng)
✓ Hệ thống sẵn sàng cho kiểm thử chấp nhận
```

Kiểm thử tích hợp hệ thống (System integration testing):

Tập trung vào việc kiểm thử các giao diện giữa hệ thống đang được kiểm thử với các hệ thống khác và các dịch vụ bên ngoài. Mức độ này yêu cầu môi trường kiểm thử phù hợp, lý tưởng nhất là tương tự như môi trường vận hành.

Mô tả: Kiểm thử giao diện giữa hệ thống với **các hệ thống bên ngoài**

📊 **Bảng đặc điểm đầy đủ:**

Thuộc tính	Chi tiết
Test Object	Giao diện giữa systems, packages, microservices

Thuộc tính	Chi tiết
Test Objectives	✓ Giảm rủi ro ✓ Xác minh giao tiếp với external systems ✓ Tìm lỗi trong external interfaces ✓ Đảm bảo data consistency giữa systems
Test Basis	✓ System design ✓ Architecture diagrams ✓ API documentation ✓ Integration specs
Test Objects	✓ Subsystems ✓ External services (payment gateways, APIs...) ✓ Microservices ✓ Cloud services
Typical Defects	✓ API calls sai ✓ Data format mismatch ✓ Timeout issues ✓ Authentication/Authorization failures ✓ Network errors not handled
Approach	✓ Test trong môi trường **gần giống production** ✓ Có thể cần test với real external systems
Environment	✓ Integration/Staging environment ✓ Có kết nối đến external services ✓ Lý tưởng: giống production

💡 **Ví dụ chi tiết:**

```
// System Integration Test: E-commerce với VNPay Payment Gateway
// Kiểm thử tích hợp hệ thống: Thương mại điện tử với cổng thanh toán VNP

describe('Kiểm thử tích hợp cổng thanh toán', () => {

  // Test 1: Thanh toán thành công
  test('xử lý thanh toán thành công qua VNPay', async () => {
    // Chuẩn bị
    const orderData = {
      orderId: 'ORD123',
      amount: 1000000, // 1,000,000 VND
      orderInfo: 'iPhone 15 Pro',
      returnUrl: 'https://myshop.com/payment/callback'
    };

    // Thực hiện
```

```
  // 1. Tạo URL thanh toán
  const paymentUrl = await vnpayService.createPaymentUrl(orderData);

  // Kiểm tra: Xác minh URL thanh toán đã tạo
  expect(paymentUrl).toContain('vnpay.vn');
  expect(paymentUrl).toContain(`vnp_Amount=${orderData.amount * 100}`);

  // 2. Mô phỏng người dùng thanh toán trên VNPay (trong test: dùng sar
  const paymentResult = await vnpaySandbox.simulatePayment(paymentUrl,
    cardNumber: '9704198526191432198',
    cardHolder: 'NGUYEN VAN A',
    expiryDate: '07/15',
    cvv: '123'
  });

  // 3. Xác minh callback đã nhận
  const callbackData = await waitForCallback();

  // Kiểm tra: Xác minh giao dịch thành công
  expect(callbackData.vnp_ResponseCode).toBe('00'); // Thành công
  expect(callbackData.vnp_TxnRef).toBe('ORD123');
  expect(callbackData.vnp_Amount).toBe('100000000'); // 1M * 100

  // 4. Xác minh trạng thái đơn hàng đã cập nhật trong hệ thống
  const order = await orderService.getOrder('ORD123');
  expect(order.status).toBe('PAID');
  expect(order.paymentMethod).toBe('VNPAY');
  expect(order.transactionId).toBeDefined();
});

// Test 2: Thanh toán thất bại
test('xử lý lỗi thanh toán một cách uyển chuyển', async () => {
  const orderData = {
    orderId: 'ORD124',
    amount: 1000000,
    orderInfo: 'iPhone 15 Pro'
  };

  // Mô phỏng số dư không đủ
  const paymentResult = await vnpaySandbox.simulatePayment(
    await vnpayService.createPaymentUrl(orderData),
    { simulateError: 'INSUFFICIENT_BALANCE' }
  );
```

```
    const callbackData = await waitForCallback();

    // Kiểm tra: Xác minh lỗi được xử lý
    expect(callbackData.vnp_ResponseCode).toBe('51'); // Số dư không đủ

    const order = await orderService.getOrder('ORD124');
    expect(order.status).toBe('PAYMENT_FAILED');
    expect(order.failureReason).toBe('Số dư không đủ');
  });

  // Test 3: Xử lý timeout
  test('xử lý timeout VNPay', async () => {
    const orderData = { orderId: 'ORD125', amount: 1000000 };

    // Mô phỏng timeout mạng
    await vnpaySandbox.simulateTimeout();

    // Kiểm tra: Hệ thống xử lý timeout
    const order = await orderService.getOrder('ORD125');
    expect(order.status).toBe('PAYMENT_PENDING');
    expect(order.notes).toContain('Timeout - chờ xác minh thủ công');
  });

  // Test 4: Dịch vụ VNPay không khả dụng
  test('xử lý khi dịch vụ VNPay ngừng hoạt động', async () => {
    // Mô phỏng VNPay bảo trì
    await vnpaySandbox.simulateDowntime();

    try {
      await vnpayService.createPaymentUrl({ orderId: 'ORD126' });
      fail('Lẽ ra phải ném lỗi');
    } catch (error) {
      expect(error.message).toContain('Cổng thanh toán không khả dụng');
      expect(error.retryable).toBe(true);
    }
  });
});
```

🌐 **Các hệ thống bên ngoài thường gặp:**

Loại hệ thống	Ví dụ	Test gì?
Cổng thanh toán	VNPay, Momo, PayPal	Quy trình thành công/thất bại, hoàn tiền, webhooks

Loại hệ thống	Ví dụ	Test gì?
Dịch vụ email	SendGrid, AWS SES	Gửi email, templates, khả năng gửi
Dịch vụ SMS	Twilio, AWS SNS	Gửi SMS, xác thực OTP
Lưu trữ đám mây	AWS S3, Google Cloud Storage	Tải lên/xuống file, phân quyền
Bản đồ/Vị trí	Google Maps API	Mã hóa địa lý, chỉ đường, tính khoảng cách
Đăng nhập mạng xã hội	Facebook, Google OAuth	Quy trình đăng nhập, xác thực token, thông tin người dùng
Phân tích	Google Analytics, Mixpanel	Theo dõi sự kiện, tính nhất quán dữ liệu
Hệ thống CRM	Salesforce	Đồng bộ dữ liệu, API calls

Kiểm thử chấp nhận (Acceptance testing)

Tập trung vào việc **xác nhận (validation)** và chứng minh sự sẵn sàng để triển khai, đảm bảo hệ thống đáp ứng nhu cầu kinh doanh của người dùng. Lý tưởng nhất là kiểm thử chấp nhận nên do người dùng thực hiện. Các hình thức chính bao gồm: kiểm thử chấp nhận người dùng (UAT), kiểm thử chấp nhận vận hành, kiểm thử chấp nhận theo hợp đồng/quy định, kiểm thử alpha và kiểm thử beta.

Mô tả: Validation - Xác nhận hệ thống đáp ứng nhu cầu người dùng

📊 **Bảng đặc điểm đầy đủ:**

Thuộc tính	Chi tiết
Test Object	Hệ thống hoàn chỉnh, processes, business workflows
Test Objectives	✓ **Validation** (building the right system) ✓ Chứng minh sẵn sàng triển khai ✓ Xác nhận đáp ứng nhu cầu kinh doanh ✓ Xây dựng niềm tin với stakeholders
Test Basis	✓ Business requirements ✓ User stories ✓ Business processes ✓ Regulations/Contracts
Test Objects	✓ Toàn bộ hệ thống ✓ Business processes ✓ Reports, dashboards ✓ Training materials

Thuộc tính	Chi tiết
Typical Defects	✓ Không đáp ứng nhu cầu người dùng ✓ Workflows không khớp thực tế ✓ Không tuân thủ quy định ✓ Không đáp ứng hợp đồng
Approach	✓ Black-box, experience-based ✓ Test trong điều kiện thực tế (hoặc gần giống) ✓ Focus on business value
Responsibilities	✓ **Lý tưởng: End users / Customers** ✓ Business analysts ✓ Product owners ✓ Subject matter experts
Environment	✓ Pre-production hoặc Production ✓ Real data (hoặc production-like data) ✓ Real user conditions

- 📋 5 hình thức Acceptance Testing (Chi tiết):

1. UAT (User Acceptance Testing - Kiểm thử chấp nhận người dùng):

```
Mục đích: Người dùng thực tế kiểm tra xem hệ thống có đáp ứng nhu cầu công
việc hàng ngày không

Ai làm: Người dùng cuối (End users), Người dùng nghiệp vụ (Business users)

Ví dụ: Hệ thống quản lý bệnh viện
- Bác sĩ test: Đăng ký bệnh nhân, kê đơn thuốc, xem lịch sử bệnh án
- Y tá test: Quản lý phòng bệnh, lập lịch tiêm chủng
- Kế toán test: Tạo hóa đơn, theo dõi thanh toán

Test scenarios dựa trên:
✓ Quy trình công việc thực tế
✓ Một ngày làm việc điển hình
✓ Trường hợp đặc biệt trong thực tế
```

2. Operational Acceptance Testing (OAT - Kiểm thử chấp nhận vận hành):

```
Mục đích: Kiểm tra khả năng vận hành hệ thống

Ai làm: Quản trị viên hệ thống (System administrators), Đội vận hành
(Operations team), DevOps

Test gì:
```

```
├─ Sao lưu & Khôi phục (Backup & Restore):
│  ✓ Sao lưu database thành công?
│  ✓ Khôi phục từ bản sao lưu hoạt động?
│  ✓ Thời gian sao lưu/khôi phục chấp nhận được?
│
├─ Khôi phục thảm họa (Disaster Recovery):
│  ✓ Chuyển đổi dự phòng sang server dự phòng?
│  ✓ Tính nhất quán dữ liệu sau khi chuyển đổi?
│
├─ Giám sát & Ghi log (Monitoring & Logging):
│  ✓ Logs được ghi đầy đủ?
│  ✓ Cảnh báo được gửi khi có lỗi?
│  ✓ Dashboard hiển thị các chỉ số?
│
├─ Triển khai (Deployment):
│  ✓ Triển khai mới không có thời gian ngừng?
│  ✓ Rollback nhanh khi cần?
│
└─ Bảo trì (Maintenance):
   ✓ Cập nhật dễ dàng?
   ✓ Không ảnh hưởng người dùng khi bảo trì?
```

3. Contractual Acceptance Testing (Kiểm thử chấp nhận theo hợp đồng):

```
Mục đích: Kiểm tra theo các điều khoản trong hợp đồng

Ai làm: Khách hàng (Customer), Đội pháp lý (Legal team), Đội QA

Ví dụ hợp đồng:
"Hệ thống phải có 100 tính năng được liệt kê trong Phụ lục A"

Quy trình kiểm thử:
1. Xem xét hợp đồng → Xác định tiêu chí chấp nhận
2. Tạo danh sách kiểm tra từ hợp đồng
3. Kiểm thử từng tính năng trong danh sách
4. Xác minh:
   ✓ 100/100 tính năng hoạt động? (90/100 = từ chối)
   ✓ Đáp ứng hiệu năng trong hợp đồng? (< 3s thời gian phản hồi)
   ✓ Đáp ứng tính khả dụng? (99.9% uptime)
5. Khách hàng ký duyệt nếu đạt

Thất bại = Không thanh toán / phạt
```

4. Regulatory/Compliance Acceptance Testing (Kiểm thử chấp nhận tuân thủ quy định):

```
Mục đích: Kiểm tra tuân thủ quy định pháp luật

Ai làm: Đội tuân thủ (Compliance team), Kiểm toán viên (Auditors), Chuyên gia pháp lý (Legal experts)

Ví dụ:

Ứng dụng ngân hàng (phải tuân thủ quy định Ngân hàng Nhà nước):
├─ KYC (Biết khách hàng của bạn - Know Your Customer):
│  ✓ Xác minh CMND/CCCD với cơ sở dữ liệu công an?
│  ✓ Nhận diện khuôn mặt chính xác?
│  ✓ Lưu trữ thông tin KYC đầy đủ?
│
├─ Chống rửa tiền (Anti-Money Laundering - AML):
│  ✓ Phát hiện giao dịch đáng ngờ?
│  ✓ Cảnh báo khi giao dịch > 500 triệu?
│  ✓ Báo cáo định kỳ cho cơ quan chức năng?
│
└─ Bảo mật dữ liệu (Data Privacy - GDPR nếu EU):
   ✓ Người dùng có thể xóa dữ liệu cá nhân?
   ✓ Dữ liệu được mã hóa?
   ✓ Chính sách bảo mật rõ ràng?

Ứng dụng y tế (phải tuân thủ quy định Y tế):
├─ HIPAA (US) / Bảo vệ dữ liệu bệnh nhân:
│  ✓ PHI (Thông tin sức khỏe được bảo vệ) được mã hóa?
│  ✓ Kiểm soát truy cập nghiêm ngặt?
│  ✓ Nhật ký kiểm toán đầy đủ?
│
└─ Quy định thiết bị y tế:
   ✓ Đã kiểm thử lâm sàng?
   ✓ Phân tích rủi ro hoàn tất?
   ✓ Tài liệu đầy đủ?
```

5. Alpha & Beta Testing (Kiểm thử Alpha & Beta):

```
Kiểm thử Alpha:
├─ Khi nào: Trước khi phát hành ra công chúng
├─ Ở đâu: Tại văn phòng của nhà phát triển
├─ Ai làm: Người dùng nội bộ (Internal users - nhân viên, team khác trong công ty)
```

```
├─ Môi trường: Phòng thí nghiệm/Môi trường test
└─ Mục đích:
   ✓ Tìm lỗi trước khi giao cho người dùng thực
   ✓ Kiểm thử trong điều kiện được kiểm soát
   ✓ Phản hồi nhanh

Ví dụ: Microsoft Office Alpha
- Nhân viên Microsoft dùng Office mới để làm việc hàng ngày
- Báo cáo lỗi qua hệ thống nội bộ
- Nhanh chóng sửa và kiểm thử lại

Kiểm thử Beta:
├─ Khi nào: Sau Alpha, trước phát hành chính thức
├─ Ở đâu: Môi trường thực tế của người dùng
├─ Ai làm: Người dùng bên ngoài (External users - khách hàng thực, người
dùng sớm)
├─ Môi trường: Production hoặc gần production
└─ Mục đích:
   ✓ Kiểm thử trong điều kiện thực tế
   ✓ Phản hồi từ người dùng thực
   ✓ Tìm lỗi không phát hiện được trong phòng thí nghiệm

Ví dụ: Chương trình iOS Beta
- Apple phát hành iOS beta cho 10000 người dùng
- Người dùng sử dụng trên iPhone thật, trong cuộc sống hàng ngày
- Báo cáo lỗi qua ứng dụng Feedback
- Apple sửa trước khi phát hành chính thức

Open Beta vs Closed Beta:
├─ Closed Beta: Giới hạn số lượng (cần mã mời)
│  Ví dụ: Gmail beta ban đầu (chỉ có mã mời)
└─ Open Beta: Ai cũng có thể tham gia
   Ví dụ: Game beta trên Steam
```

Để tránh sự chồng chéo của các hoạt động kiểm thử, các mức độ kiểm thử được phân biệt bởi các thuộc tính sau:

- Đối tượng kiểm thử.
- Mục tiêu kiểm thử.
- Cơ sở kiểm thử.
- Khuyết tật và thất bại.
- Phương pháp tiếp cận và trách nhiệm.

Để **tránh sự chồng chéo** giữa các mức độ, chúng được phân biệt bởi:

Thuộc tính	Giải thích	Ví dụ
1️⃣ **Test Object**	Đối tượng được kiểm thử	Component: Module System: Toàn bộ ứng dụng
2️⃣ **Test Objectives**	Mục tiêu kiểm thử	Component: Logic đúng Acceptance: Đáp ứng nhu cầu
3️⃣ **Test Basis**	Cơ sở để viết test	Component: Code/Design System: Requirements
4️⃣ **Defects & Failures**	Loại lỗi tìm thấy	Component: Logic errors System: Integration issues
5️⃣ **Approach & Responsibilities**	Cách tiếp cận và ai làm	Component: Dev, White-box Acceptance: Users, Black-box

---

📊 So sánh 5 Test Levels (Bảng tổng hợp):

Aspect	Component	Component Integration	System	System Integration	Acceptance
Scope	Single unit	Multiple units	Whole system	System + External	Business value
Tester	Developer	Dev/Tester	Independent QA	QA + DevOps	End users
Focus	Logic, code	Interfaces	End-to-end	External APIs	User needs
Test Basis	Code, design	Architecture	Requirements	API docs	Business requirements
Test Type	Mainly white-box	White/Black mix	Black-box	Black-box	Black-box
Environment	Dev machine	Dev/Test	Test env	Staging	Pre-prod/Prod
Defects Found	Logic errors	Integration errors	Functional/Non-functional	API/Integration errors	Usability, business issues
Automation %	90%+	70-80%	50-70%	40-60%	20-40%
When	During coding	After units ready	After integration	After system testing	Before release

---

💡 Ví dụ tổng hợp qua 5 levels:

Tính năng: Đăng ký người dùng

```
1️⃣ Kiểm thử thành phần (Component Testing):
   Test: Hàm validateEmail()
   ├─ Đầu vào: "test@example.com" → Hợp lệ ✓
   ├─ Đầu vào: "invalid-email" → Không hợp lệ ✓
   └─ Đầu vào: "" → Lỗi trống ✓

2️⃣ Kiểm thử tích hợp thành phần (Component Integration Testing):
   Test: Form đăng ký + Validation + Database
   ├─ Form gửi dữ liệu đến Validator
   ├─ Validator kiểm tra và trả về kết quả
   ├─ Nếu hợp lệ, lưu vào Database
   └─ Trả về thành công/lỗi cho Form

3️⃣ Kiểm thử hệ thống (System Testing):
   Test: Quy trình đăng ký hoàn chỉnh end-to-end
   ├─ Điều hướng đến /register
   ├─ Điền form (tên, email, mật khẩu)
   ├─ Gửi form
   ├─ Xác minh email đã gửi
   ├─ Nhấp link xác thực
   ├─ Xác minh tài khoản đã kích hoạt
   └─ Xác minh có thể đăng nhập

4️⃣ Kiểm thử tích hợp hệ thống (System Integration Testing):
   Test: Tích hợp với dịch vụ Email (SendGrid)
   ├─ Đăng ký kích hoạt email qua SendGrid API
   ├─ Xác minh email đã gửi
   ├─ Xác minh template email đúng
   └─ Xử lý lỗi SendGrid một cách uyển chuyển

5️⃣ Kiểm thử chấp nhận (Acceptance Testing - UAT):
   Test: Góc độ nghiệp vụ
   ├─ Đội Marketing: Có lấy được dữ liệu người dùng mới không?
   ├─ Đội Sales: Có ghi nhận nguồn khách hàng không?
   ├─ Đội Pháp lý: Chính sách bảo mật có hiển thị không?
   └─ Người dùng cuối: Đăng ký có dễ không? < 2 phút?
```

📚 Tóm tắt Key Points:

🔑 Phân biệt Test Levels:

5 thuộc tính phân biệt:

1. Test Object (test cái gì)
2. Test Objectives (mục tiêu gì)
3. Test Basis (dựa vào đâu)
4. Defects & Failures (lỗi gì)
5. Approach & Responsibilities (ai làm, làm như thế nào)

🔗 Mối quan hệ giữa các levels:

- **Sequential SDLC:** Exit criteria của level này = Entry criteria của level sau
- **Iterative SDLC:** Các levels có thể chồng chéo thời gian

⚡ Điểm quan trọng:

- **Component:** Dev làm, white-box, test logic
- **Integration:** Test giao diện, phụ thuộc strategy (bottom-up/top-down/big-bang)
- **System:** Independent QA, black-box, end-to-end
- **System Integration:** Test với external systems, cần môi trường gần production
- **Acceptance:** Users làm, validation, chứng minh sẵn sàng deploy